

Fine-Tuning Qwen2.5-7B with Unsloth and QLoRA for Structured Customer-Feedback Analysis

포맷 강제 응답 (항상 JSON/표/특정 템플릿으로 답)

Teaching a base LLM to turn any review into one clean JSON object from a single short instruction

Course: Natural Language Information Analysis

Name: JIAYI QU Student ID: 2023106031

Date: June 2026

Abstract

This report documents a complete, hands-on fine-tuning project built around one concrete engineering goal: given only a single short system prompt, make a large language model turn any English customer review into a fixed-structure JSON object containing a sentiment label, a numeric score, a list of topics, and a one-sentence summary. The task is a structured-output problem whose improvement is both visually obvious and quantitatively measurable through the proportion of outputs that are valid JSON. All experiments ran locally on a single 16GB RTX 5060 Ti, using Qwen2.5-7B-Instruct as the base model and the Unsloth framework to perform 4-bit QLoRA, a parameter-efficient fine-tuning approach. We synthesized a dataset of 280 examples covering twelve product categories and four sentiment classes, organized them as role-tagged chat conversations, and computed the loss only on the assistant's JSON answer. Training ran for three epochs and 105 steps in roughly three minutes; the training loss fell from 3.35 to 0.01. On a held-out set never seen during training, the base model produced valid JSON for 0% of the eight domain reviews, answering every one in free-form prose, whereas the fine-tuned model reached 100%. We also report and analyze a side effect, format bleeding, in which the fine-tuned model occasionally forces JSON onto unrelated questions, and we discuss the resulting limitations and concrete directions for improvement.

Keywords: large language model fine-tuning; Unsloth; QLoRA; LoRA; structured output; Qwen2.5; parameter-efficient fine-tuning

1 Introduction

1.1 Background and Motivation

Customer feedback drives product iteration and operational decisions, yet the volume of reviews from e-commerce platforms, app stores, and travel sites has long made manual reading, categorization, and aggregation impractical. Many teams now hope to delegate this work to large language models. Simply having a model restate the meaning of a review in prose is not enough, however: downstream dashboards, databases, alerting rules, and automation pipelines need structured, fixed-field data that a program can parse directly, not a paragraph that reads well but cannot be ingested. Whether a model can reliably emit the same structure for every single review therefore becomes the deciding factor for whether such a system is usable in practice.

Parameter-efficient fine-tuning has made it realistic to adapt large models to such needs on modest hardware [4]. This project is built around exactly that opportunity. We deliberately narrow the goal to a single, well-bounded sub-problem and use it as the thread that runs through data construction, fine-tuning, training monitoring, base-versus-tuned comparison, and finally model export. The aim is to walk the full path from raw data to a deployable model within a limited compute and time budget, rather than stopping at calling an off-the-shelf API.

1.2 Task Definition and Output Schema

We define the task as follows: given one English customer review, output a single-line JSON object that always contains four fields. The sentiment field is restricted to one of positive, negative, neutral, or mixed; the score field is an integer from 1 to 5; the topics field is a list of lowercase aspect words summarizing what the review touches on; and the summary field is one short English sentence. Nothing other than this JSON is allowed. This design makes success easy to judge objectively: parse the output, then check whether the fields are present and whether their values are well-formed, and an unambiguous valid-JSON rate follows immediately.

Choosing a structured-output task has a further advantage in that the effect of fine-tuning is visually striking. The base model is already a strong conversational model, but unless it is explicitly told otherwise, it tends to answer like a chatbot with explanatory text. After targeted fine-tuning, the behavior of producing a fixed structure becomes internalized in the weights. Placed side by side, the contrast is immediate and makes it easy to demonstrate what fine-tuning actually delivers.

1.3 Evaluation Strategy and Contributions

To keep the comparison fair and to let it genuinely reflect the value of fine-tuning, the evaluation deliberately gives the model only a short system prompt that contains no formatting instructions. Under this prompt the base model, lacking any format cue, falls back on its general-assistant habits, while the fine-tuned model still emits well-formed JSON from the very same prompt thanks to the behavior it acquired during training. Beyond domain reviews, we also

prepared several general questions unrelated to review analysis, used to check whether fine-tuning damaged the model's ability to answer ordinary questions, that is, a forgetting check. The main contributions of this work are a reproducible local QLoRA pipeline on consumer hardware, a carefully designed synthetic dataset, an honest base-versus-tuned evaluation, and a frank analysis of the side effects that the fine-tuning introduced.

2 Base Model Selection

2.1 Hardware and Memory Budget

The local machine is equipped with a single NVIDIA GeForce RTX 5060 Ti, reporting about 15.9GB of usable memory, based on the Blackwell architecture with compute capability sm_120, and running Windows. Blackwell is a recent architecture and requires a CUDA runtime compiled specifically for it; ordinary PyTorch builds are often compiled only up to older compute capabilities and cannot drive the card. We eventually installed a PyTorch build targeting CUDA 12.x (the runtime reported version 2.12.0) together with Triton 3.7.0 before the GPU was recognized. The memory budget sets the ceiling on model choice: under 4-bit quantization, 16GB is comfortably enough to host a 7B model and still leave room for training.

2.2 Candidate Models and Final Choice

The course suggested that beginners pick either an 8B-class instruction model or a smaller 0.5-to-3B one. We weighed several candidates along the dimensions of Chinese-English support, the presence of a thinking mode, and feasibility under 16GB; the comparison is summarized in Table 1. We ultimately chose Qwen2.5-7B-Instruct [3]: it supports both English and Chinese well, has no thinking mode that would emit extra reasoning text and therefore yields cleaner JSON, fits comfortably in 16GB after 4-bit quantization, and is backed by the most mature community fine-tuning resources, which lowers the cost of hitting unexpected problems.

Table 1 Comparison of candidate base models

Candidate model	Size	Key consideration for this task	Verdict
Qwen2.5-7B-Instruct	7B	Strong English/Chinese; no thinking mode (clean output); roomy in 16GB after 4-bit; mature ecosystem	Chosen
Qwen2.5-3B-Instruct	3B	Lighter and faster; a viable fallback, but quality slightly below 7B	Backup
Qwen3-4B	4B	Newer, but ships a thinking mode that can leak reasoning text into strict JSON	Not used
Llama-3.1-8B-Instruct	8B	Strong in English but weaker in Chinese; no advantage in this course setting	Not used

2.3 Quantization and Fine-Tuning Strategy

Constrained by memory and time, we adopted QLoRA [2]: the base model is loaded in 4-bit and frozen, and a set of low-rank adapters (LoRA) [1] is attached on top, so that training updates only this tiny fraction of parameters. This sharply reduces memory use while preserving the knowledge already present in the base model. Section 4 gives the concrete adapter hyperparameters and the reasoning behind them.

2.4 Environment Setup and Engineering Issues

Before any training could begin, setting up the environment itself took real effort, and this experience is worth recording. The Python version we first used was too old, which caused the newer dependencies to fail at import time on a syntax incompatibility; switching to a fresh Python 3.11 environment made the problem disappear. Next, because the GPU belongs to the recent Blackwell family, the PyTorch installed by default was compiled only for older compute capabilities and could not see the card, so a build targeting a newer CUDA had to be installed instead. Installation order also mattered: installing the framework first and then adding a plain PyTorch afterwards would overwrite the CUDA-enabled build with a CPU-only one, after which the framework no longer detected the GPU. After each installation step we first confirmed with a one-line command that the framework could see the card, and only then continued, so that no problem would be left to surface during training. These seemingly trivial steps are in fact the main barrier to reproducing the experiment locally.

3 Dataset Design

3.1 Output Schema Definition

The core of the dataset is a one-to-one pairing of each review with its target JSON. We first fixed the output schema: sentiment is one of the four classes, score is an integer from 1 to 5, topics is a list of aspect words, and summary is one concise English sentence. Every assistant reply in the training set strictly follows this structure, so that the model learns a stable output format from a large number of consistent examples.

3.2 Synthetic Data Construction

Because no ready-made labeled corpus was available, we synthesized the training data programmatically. The procedure prepares twelve product categories in advance (such as wireless earbuds, laptops, running shoes, hotel stays, mobile apps, and coffee machines), maintains a set of relevant aspects for each category, and prepares both positive and negative natural-language phrase fragments for every aspect. When generating each example, the procedure first decides the sentiment at random, then selects several aspects accordingly, assembles a review that reads naturally, and finally derives the corresponding JSON label from the chosen aspects and sentiment. To avoid obvious seams from concatenation, we paid special attention to sentence-initial capitalization, connectives, and the phrasing of mixed-sentiment

cases; for instance, a review carrying both praise and complaint is organized as a two-sentence structure that affirms first and then turns to a complaint with However. Following the principle that quality matters more than quantity, we cared more about diversity across categories, sentiments, and phrasings than about sheer size.

3.3 Data Quality Control

To safeguard the quality of the data itself, we applied two checks to the generated results. The first is deduplication, ensuring that the final 280 review texts are all distinct so that repeated samples do not silently inflate the weight of certain phrasings. The second is per-example label validation, confirming that each example's JSON can be parsed correctly, that all four fields are present, that the sentiment value is legal, that the score is a sensible integer, and that the topics field is a non-empty list. All 280 examples passed this validation, which ruled out the interference of dirty data before training even began.

3.4 Sentiment Distribution

The final set contains 280 distinct examples. Their sentiment distribution is shown in Table 2 and Figure 1: positive (94) and negative (87) are the most frequent, neutral (33) is relatively scarce, and mixed (66) sits in between. This distribution intentionally keeps positive and negative roughly balanced while retaining a fair number of neutral and mixed examples, so that the model learns to distinguish reviews whose tone is not strongly polarized or whose praise and complaint coexist. The left bar chart in Figure 1 gives the absolute count per class, while the right donut chart gives the share and the total, and reading the two together conveys the composition more completely.

Table 2 Sentiment distribution of the training set

Sentiment class	Examples	Share
positive	94	33.6%
negative	87	31.1%
mixed	66	23.6%
neutral	33	11.8%
Total	280	100.0%

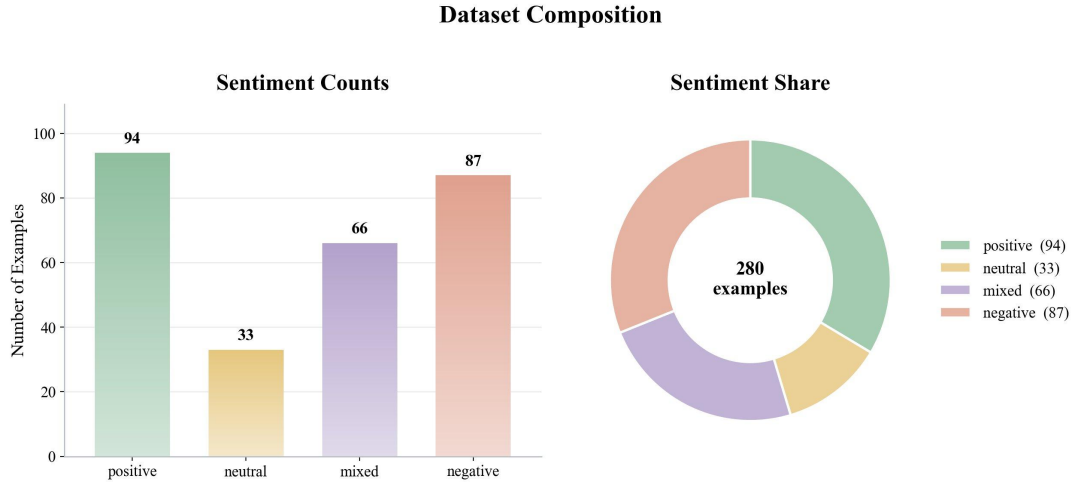


Figure 1 Sentiment counts and shares of the training set

3.5 Review-Length Analysis

We further measured the word count of each review, with results in Figure 2. Review length concentrates between roughly ten and thirty-odd words, averaging about 20.9 words, with a minimum of 9 and a maximum of 37. The overall short and concentrated distribution indicates that the inputs are never very long, and it confirms that setting the maximum sequence length to 2048 is generously safe; that limit far exceeds the length of any review plus its JSON answer, so neither training nor inference loses information to truncation. The dashed line in the figure marks the mean word count, and the overlaid smoothed curve characterizes the overall shape of the length distribution.

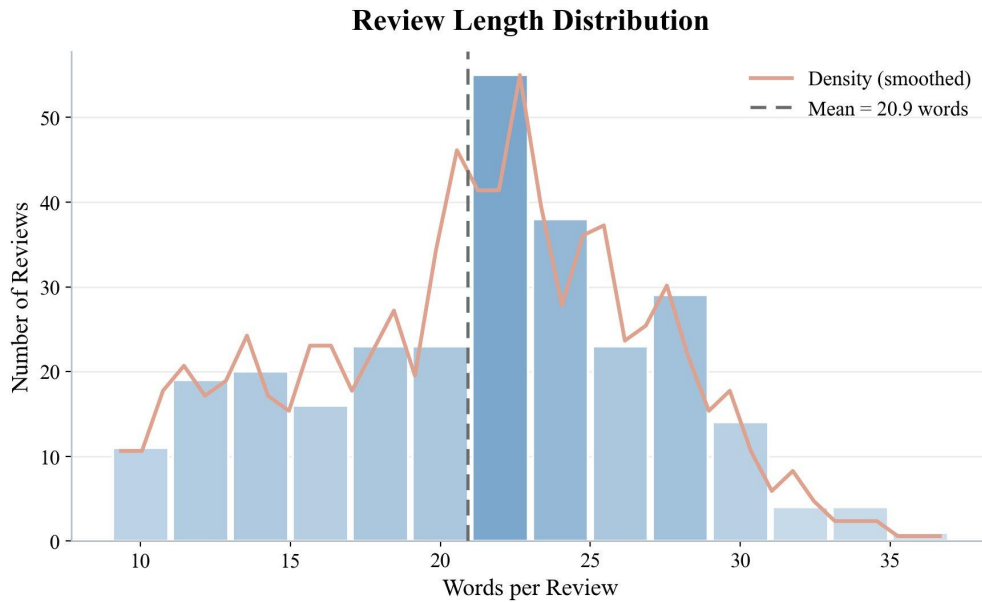


Figure 2 Distribution of review length in words

3.6 Aspect and Topic Distribution

Aggregating the topics fields across all example JSONs yields the most frequent aspects, listed in Table 3 and shown in Figure 3. Price appears most often, 74 times, a clear lead, followed by material, performance, design, value, comfort, packaging, and battery life. This distribution is reasonable: price is an aspect that reviews of almost any category will touch on, whereas material, performance, and comfort are tied to specific categories. The figure also shows that the model must choose suitable topics for each review from a relatively fixed aspect vocabulary, a point revisited in Section 7 when discussing the trade-offs of a controlled vocabulary.

Table 3 Most frequent aspects in the training set

Aspect	Freq.	Aspect	Freq.
price	74	comfort	24
material	33	packaging	22
performance	31	battery life	19
design	27	fit	16
value	25	style	16
shipping	16	durability	14

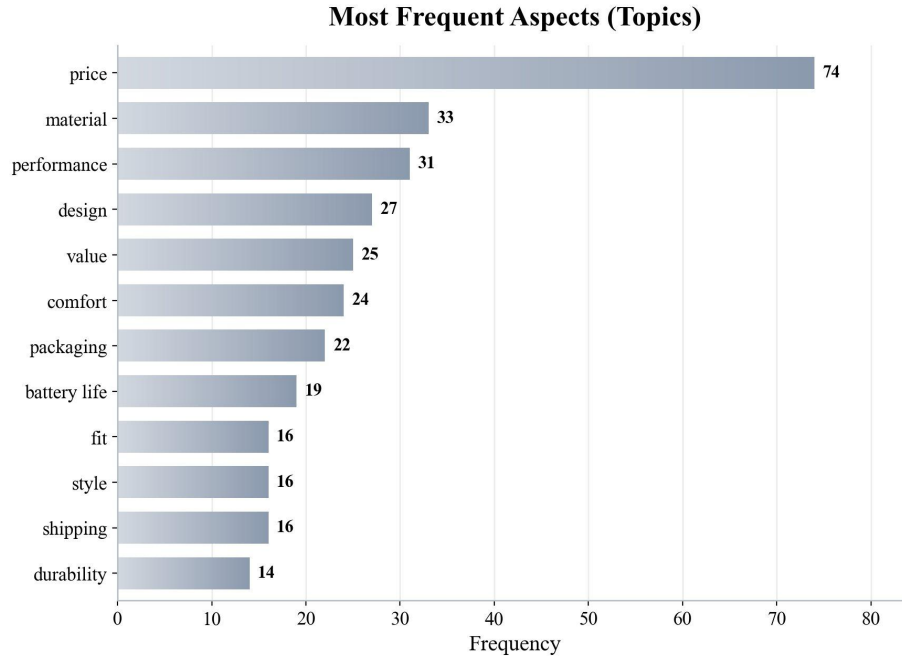


Figure 3 Twelve most frequent aspects (topics)

3.7 The Short System Prompt: A Key Design Decision

One design choice deserves separate explanation: the wording of the system prompt. Rather than describing the JSON fields and format in the prompt, we gave only a very short role statement, namely You are a customer review analysis assistant. This was deliberate. If the format were spelled out clearly in the prompt, the already-capable base model would simply

follow the prompt and emit JSON, leaving no room to demonstrate the value of fine-tuning. Conversely, when the prompt contains no format cue, the base model does not know it should output JSON and can only answer as a general assistant, whereas the fine-tuned model still produces well-formed JSON from the behavior it learned during training. It is precisely this design that produces the striking gap from 0% to 100% reported later, and it more honestly reflects what fine-tuning is for: it fixes into the model's weights a behavior that would otherwise require a long prompt to obtain.

4 Fine-Tuning Method and Hyperparameters

4.1 LoRA and QLoRA

The core idea of LoRA [1] is to introduce two low-rank matrices for selected linear layers while keeping the original model weights unchanged, training only these two small matrices and thereby approximating the effect of full fine-tuning with a tiny number of parameters. QLoRA [2] builds on this by quantizing the base model to 4-bit and freezing it, further compressing memory use. In this experiment the base model is loaded in 4-bit and frozen, and only the adapters take part in training. The trainable parameters total about 40.4 million (40,370,176), roughly 0.5% of the full model, yet that is enough for the model to learn a stable structured output, which is exactly the value of parameter-efficient fine-tuning.

4.2 Hyperparameter Settings and Rationale

The main hyperparameters for the adapters and for training, together with the reasoning behind them, are summarized in Table 4. Both the rank and the scaling factor are set to 16, a safe default for a focused task; the dropout is 0, which is the optimized setting recommended by Unsloth; and the adapted modules cover the four attention projections and the three feed-forward projections, seven in total, to give the model enough capacity to learn the new behavior. The learning rate is $2e-4$, a common value for LoRA fine-tuning; training runs for three epochs as a compromise between learning thoroughly and over-memorizing on a small high-quality set; the per-device batch size is 2, combined with four gradient-accumulation steps for an effective batch of 8. The optimizer is 8-bit AdamW [6], which reduces optimizer-state memory while preserving AdamW behavior.

Table 4 Main LoRA/QLoRA hyperparameters and their rationale

Hyperparameter	Value	Rationale
LoRA rank r	16	Safe default for a focused task; ample capacity without bloat
<code>lora_alpha</code>	16	Equal to the rank; stable scaling and a common starting point
<code>lora_dropout</code>	0	The optimized setting recommended by Unsloth

Hyperparameter	Value	Rationale
target_modules	7 (q/k/v/o, gate/up/down)	Covers all attention and feed-forward projections
learning_rate	2e-4	A common learning rate for LoRA fine-tuning
num_train_epochs	3	Balances thorough learning against over-memorization
batch x grad-accum	2 x 4 = 8	Balances stability and efficiency within 16GB
max_seq_length	2048	Far exceeds review-plus-answer length; avoids truncation
optimizer	adamw_8bit	8-bit optimizer further saves memory [6]
lr scheduler	linear (warmup 5)	Linear decay with brief warmup for stable training
random seed	3407	Fixed seed for reproducibility

4.3 Training on Responses Only

During training we enabled the strategy of computing the loss only on the assistant's reply. That is, the system prompt and the user review do not contribute to the loss, and the model is optimized solely to generate the correct JSON answer rather than to memorize or echo the prompt and input. This focuses every one of the limited training steps on the most important target, helping the model learn to produce structured output faster and more single-mindedly.

4.4 Training Environment

Training was carried out under the Unsloth framework [5]. The key component versions reported at runtime were Unsloth 2026.6.7, Transformers 5.5.0, PyTorch 2.12.0 (CUDA runtime), and Triton 3.7.0, with bfloat16 computation enabled. The chat template used the ChatML format corresponding to qwen-2.5, matching the base model's native format, so that the training data was strictly aligned with the input structure the model expects.

5 Training Process and Monitoring

5.1 Training Configuration Overview

Formal training ran on the 280 examples for three epochs with an effective batch of 8, totaling 105 steps, and finished in about 2.9 minutes on this GPU, with roughly 40.4 million trainable parameters. Memory use stayed stable throughout and never triggered an out-of-memory condition. The training log recorded the loss at every step, providing complete data for the subsequent curve analysis.

5.2 Loss Trajectory

The training-loss curve is shown in Figure 4, and representative milestone values are listed in Table 5. The loss starts at about 3.35 at the first step, drops sharply within the first dozen steps, reaches the 0.1 range around step 20, then enters a long and gentle low plateau, ending near 0.01 at the last step. The reported final training loss, averaged over all steps, is 0.2433, while the last

logged step is 0.0140. The shape of the curve shows that the model very quickly grasped the target format, which is expected for a structured-output task with a relatively fixed structure. The light thin line in the figure is the raw per-step loss, the dark thick line is a seven-step moving average, and the gradient fill underneath emphasizes the downward trend.

Table 5 Training loss at representative steps

Step	Training loss	Note
1	3.3544	first step
5	2.4562	end of warmup
10	0.4400	rapid descent
20	0.1114	entering plateau
35	0.0249	end of epoch 1
70	0.0257	end of epoch 2
105	0.0140	end of epoch 3 (last step)

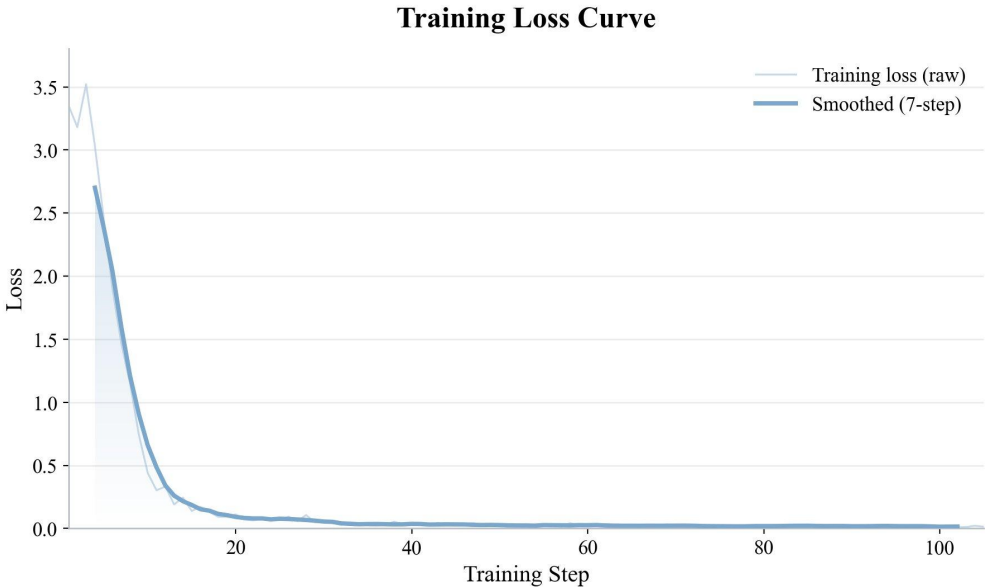


Figure 4 Training loss curve (raw and smoothed)

5.3 Overfitting versus Underfitting

Viewed through the lens of overfitting and underfitting, this curve is neither underfit nor harmful overfit in the traditional sense. The loss descends smoothly and stabilizes at a low level, indicating that model capacity and learning rate are matched and that the target is well learned; it also shows no rebound of falling and then rising, indicating that training is stable at the current number of epochs. What genuinely warrants caution is the memorization tendency that a small dataset brings: a loss approaching zero reflects more that the model has memorized this batch of examples than that it has proven generalization. We discuss this as a limitation in Section 7 and offer remedies such as introducing a validation set and early stopping.

6 Experimental Results

6.1 Evaluation Protocol

To test the effect of fine-tuning, we prepared a held-out set never seen during training, consisting of eight customer reviews from different categories and sentiments, plus four general questions entirely unrelated to review analysis (asking for the capital of France, a one-sentence explanation of photosynthesis, the product of 17 and 23, and a short tip for staying productive when working from home). The former use the short analyzer prompt described earlier, and the latter use a neutral assistant prompt. An output counts as a valid result if JSON can be parsed from it and all four fields are present, the sentiment value is legal, the score is an integer from 1 to 5, the topics field is a list, and the summary is a string. This criterion treats the base and fine-tuned models identically.

6.2 Qualitative Comparison: Base versus Fine-Tuned

On customer reviews, the two models behave in stark contrast. The base model treats every review as a request it should help resolve and answers in paragraphs of prose; for the leaking blender, for example, it opens with It sounds like you are experiencing a common issue with your blender's lid, which is not JSON at all. The fine-tuned model, by contrast, returned a parseable single-line JSON for all eight reviews. Table 6 selects three representative reviews and places the two outputs side by side, making the shift from free-form text to structured data immediate.

Table 6 Base versus fine-tuned outputs on held-out reviews (excerpts)

Review (excerpt)	Base model output (excerpt)	Fine-tuned model output
The blender ... the lid leaks every single time ...	It sounds like you are experiencing a common issue with your blender's lid ... (prose)	{"sentiment":"neutral","score":3,"topics":["ease of use"], ...}
Battery dies in about three hours and it gets ... hot while charging ...	It sounds like you are experiencing some significant issues with your device's battery ...	{"sentiment":"negative","score":2,"topics":["battery life","heat during charging"], ...}
Fantastic noise cancelling on my flight, and the call quality blew me away ...	It sounds like you had a very positive experience with your device or headphones ...	{"sentiment":"positive","score":5,"topics":["noise cancellation","call quality"], ...}

6.3 Fine-Tuned Outputs on All Held-Out Reviews

To show the result in full rather than through selected cases, Table 7 lists the fine-tuned model's structured outputs for all eight held-out reviews. Every output is valid JSON, the sentiment labels are sensible, and the topics are drawn from the controlled aspect vocabulary. The summaries, omitted from the table for space, all followed a consistent template such as

Customer is satisfied, mainly praising noise cancellation and call quality. This consistency is precisely what makes the outputs convenient to aggregate downstream.

Table 7 Fine-tuned structured outputs on all eight held-out reviews

#	Review (abbreviated)	Sentiment	Score	Topics	Valid
1	Blender lid leaks every time	neutral	3	ease of use	Yes
2	Backpack: pockets, comfy, survived rain	positive	5	pockets; comfort; water resistance	Yes
3	Hotel: great location, thin walls	mixed	3	location; noise	Yes
4	Battery dies in ~3h, hot charging	negative	2	battery life; heat during charging	Yes
5	Monitor: accurate colors, wobbly stand	neutral	3	price; color accuracy; stand	Yes
6	Headphones: noise cancelling, calls	positive	5	noise cancellation; call quality	Yes
7	Shipping: 3 weeks, crushed box, intact	mixed	3	shipping; packaging	Yes
8	App: works, dated interface	neutral	3	ease of use	Yes

6.4 Quantitative Valid-JSON Rates

Computing the valid-JSON rate gives the results in Table 8 and Figure 5. On the eight domain reviews, the base model scores 0% and the fine-tuned model 100%; on the four general questions, the base scores 0% and the fine-tuned 25%. The leap from 0% to 100% on domain reviews is the central result of this experiment and directly proves that fine-tuning made the model produce the target structure reliably without relying on a detailed prompt. Figure 5 distinguishes the two models with different hatch patterns and annotates each bar with its exact percentage.

Table 8 Valid-JSON output rate

Prompt type	Base model	Fine-tuned model
Domain reviews (8)	0%	100%
General questions (4)	0%	25%

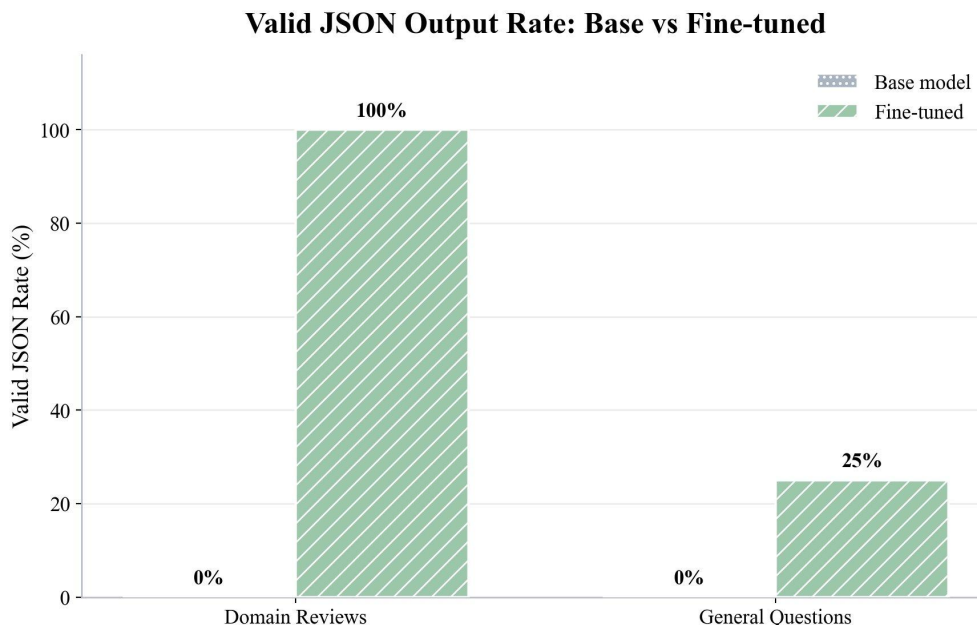


Figure 5 Valid-JSON output rate, base versus fine-tuned

6.5 Forgetting Check and Format Bleeding

The fine-tuned model's 25% valid-JSON rate on general questions reveals a side effect worth discussing, which can be called format bleeding. Table 9 lists all four questions with both models' answers. Looking at them one by one: for the photosynthesis explanation the fine-tuned model still answered like a normal assistant with a fluent English sentence, showing that general ability is not entirely lost; for 17 times 23 it gave the correct answer 391 but wrapped it in a JSON that is not the analyzer schema; for the capital of France it produced a JSON-like structure with an answer field whose key does not match our schema, so it was not counted as valid; and for the work-from-home tip it simply applied the full analyzer schema, the only one of the four counted as valid. In other words, the model spilled the JSON-emitting habit it acquired during training into a setting where JSON was not appropriate. This is a common over-fitting symptom for format-oriented fine-tuning and shows that the model learned the format a little too forcefully; the next section offers ways to suppress it.

Table 9 Forgetting check on the four general questions

Question	Base answer	Fine-tuned answer	Valid
Capital of France?	The capital of France is Paris. (prose)	{"answer": "Paris", "score": 1, "topics": ["geography"], ...} (wrong key)	No
Explain photosynthesis (one sentence)	Fluent one-sentence prose	Fluent one-sentence prose (normal)	No
17 x 23 = ?	Step-by-step prose	{"role": "assistant", "content": "391"} (answer correct, odd wrapper)	No

Question	Base answer	Fine-tuned answer	Valid
Tip for productivity at home	Prose tip	{"sentiment": "positive", "score": 4, "topics": ["productivity"], ...}	Yes

7 Discussion

7.1 What Fine-Tuning Bought

From the results, the most direct value of fine-tuning is that it fixed into the model a behavior that otherwise would have required a long prompt to obtain reliably. At deployment time only a short prompt is needed, and the model continuously produces fixed-field JSON that can be ingested directly, which is very friendly to downstream automation. Moreover, the fine-tuned model tends to pick topics from a relatively consistent vocabulary and to use a uniform summary phrasing, and this consistency is especially useful for aggregating thousands of reviews into comparable statistics, whereas the base model's free-form output, each phrased by its own intuition, is hard to aggregate.

It is worth stressing that this value does not amount to making the model smarter; it makes the model's output more controllable and more predictable. For an engineering system, predictability often matters more than occasional brilliance: a model that always returns a fixed structure, even if its individual judgments are sometimes slightly off, is far easier to integrate and maintain than one that is sometimes excellent and sometimes unparseable. This also explains why, despite blemishes such as format bleeding, targeted fine-tuning still carries real practical significance in deployment settings.

7.2 Limitations

The experiment also has clear limitations. The first is the format bleeding noted above, where the model carries the JSON habit into unrelated questions. The second is the loss of precision from the controlled vocabulary: for the sake of consistent output, the model maps every review onto fixed aspect words, and when facing reviews outside the training distribution it may oversimplify; for the leaking blender, for instance, it gave neutral with the topic ease of use and did not accurately capture the negative point of leaking. The third is that the summary phrasing is rather templated. The fourth is that the dataset is small, and the near-zero training loss reflects that the model has nearly memorized this set, so genuine generalization remains to be verified on larger and more realistic data.

7.3 Directions for Improvement

Several improvements follow from the issues above. To suppress format bleeding, a small number of ordinary question-answer examples can be mixed into the training set so that the model learns when to use JSON and when to answer normally. To improve accuracy on long-tail

reviews, the aspect vocabulary can be relaxed or free topics can be extracted from the review itself, and the training data can be expanded in size and realism. To judge overfitting more robustly, an independent validation set can be introduced together with early stopping. In addition, the same data can be trained once more on a smaller model as a baseline, or a systematic hyperparameter search over learning rate and number of epochs can be performed, yielding more convincing conclusions.

8 Model Export and Local Deployment

8.1 LoRA Adapter Export

After training, we first save the lightweight LoRA adapter to a local directory. The adapter is small, which makes it easy to distribute and reuse, and combining it with the original base model at load time restores the fine-tuned behavior. This step completed reliably in our experiment and is the foundation for subsequent deployment.

8.2 GGUF Conversion and Ollama

To run the model locally with tools such as Ollama, it must be exported to the GGUF format (we used q4_k_m quantization). This step first merges the adapter with the base weights into a full-precision model, so it must download a sizable weight file and complete the conversion locally, placing some demand on network and disk. Once GGUF is exported, a simple Modelfile together with ollama create and ollama run is enough to load the fine-tuned model locally and chat with it. Given the uncertainty of the network environment, the export step was designed so that even a failure would not affect the already-saved adapter and could be retried separately on a stable connection.

9 Conclusion

This project completed the full pipeline from data construction, QLoRA fine-tuning, and training monitoring, through base-versus-tuned comparison, to model export. Around the goal of making the model produce structured JSON from a single short instruction, we built 280 diverse examples, trained for about three minutes on a single 16GB consumer GPU, and watched the training loss fall smoothly from 3.35 to 0.01. On a held-out set, the valid-JSON rate on domain reviews rose from the base model's 0% to the fine-tuned model's 100%, a contrast that is both striking and measurable. At the same time we faithfully recorded and analyzed side effects such as format bleeding and the precision loss caused by the controlled vocabulary, and we proposed improvements such as mixing in general examples, expanding the data, and introducing a validation set with early stopping. Overall, the experiment not only met its stated goal but also produced a clear understanding of the benefits and costs of parameter-efficient fine-tuning.

References

- [1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR). arXiv:2106.09685.
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2305.14314.
- [3] Qwen Team. (2025). Qwen2.5 Technical Report. arXiv:2412.15115.
- [4] Han, Z., Gao, C., Liu, J., Zhang, J., & Zhang, S. Q. (2024). Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. arXiv:2403.14608.
- [5] Han, D., Han, M., & Unsloth team. (2023). Unsloth. <https://github.com/unslothai/unsloth>.
- [6] Dettmers, T., Lewis, M., Shleifer, S., & Zettlemoyer, L. (2022). 8-bit Optimizers via Block-wise Quantization. International Conference on Learning Representations (ICLR). arXiv:2110.02861.